

Logistic Regression

Breast Cancer Classification

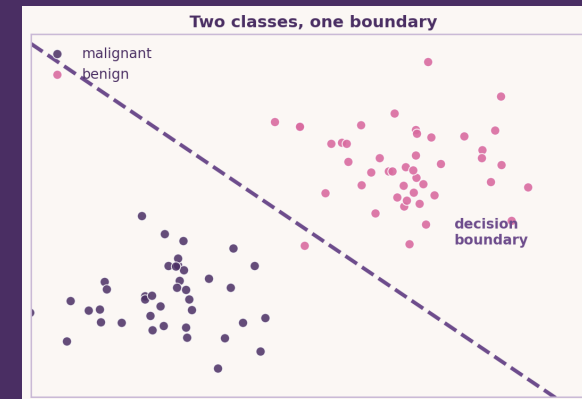
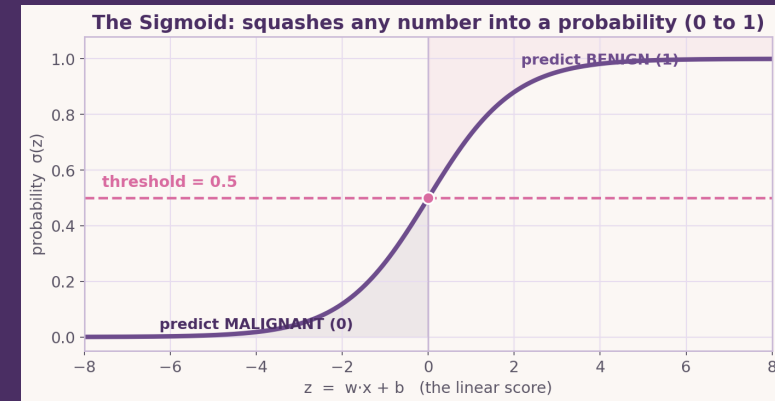
Theory + Code · beginner friendly

Created by Nikita Mishra

Python

pandas

scikit-learn



DIGITAL GYAAN

PART 1

The Theory

samajhte hain, ekdum simple language mein

What is Logistic Regression?

Naam mein “regression” hai, par ye asal mein ek **classification** algorithm hai — yes/no type sawaalon ke liye.

- It outputs a **probability between 0 and 1** (e.g. 0.92 = 92% chance benign).
- Then it picks the **most likely class** using a threshold (usually 0.5).
- Here the question is: is this tumor **malignant (0)** or **benign (1)**?

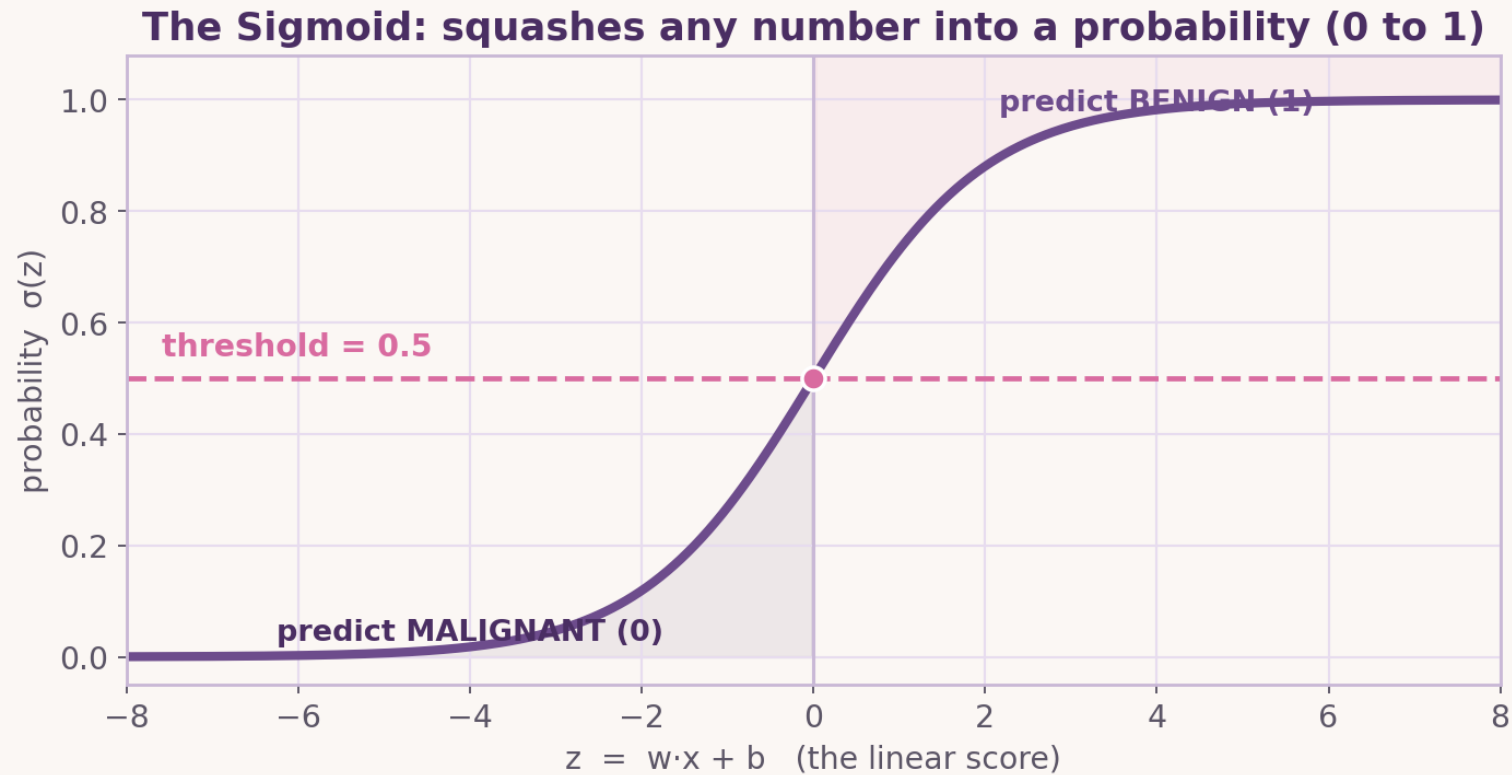
The problem we're solving

Hamare paas ek tumor ki **measurements** hain — radius, texture, smoothness, waghaira. In clues se hume ek **haan/naa** answer chahiye.

- **Input (X):** 30 numeric measurements of the tumor.
- **Output (Y):** a label — malignant or benign.
- A correct answer here can genuinely **matter for a patient** — so accuracy counts.

The sigmoid function

The model first computes a linear score, then the **sigmoid** squashes it into a probability between 0 and 1. Above **0.5** → benign; below → malignant.



The maths, made simple

Teen steps mein samjho:

$$z = w \cdot x + b \rightarrow p = \sigma(z) = 1 / (1 + e^{-z}) \rightarrow \text{class}$$

- $z = w \cdot x + b$ — a weighted sum of the features (the linear score).
- $\sigma(z)$ — the sigmoid turns z into a probability p .
- Training minimises **log-loss** (cross-entropy) — it punishes confident wrong answers.

Scaling & train/test split

- **Scaling (StandardScaler):** features have very different ranges; we rescale each to mean 0, std 1 so the model learns fairly and faster.
- **Fit on train, transform on test:** we learn the scaling from training data only — the test set must stay “unseen”.
- **Train/test split:** we hide 20% of the data to grade the model fairly — like not giving a student the exam answers in advance.

How we grade the model

- **Accuracy** — overall, what % of predictions were correct.
- **Precision** — of those it called positive, how many really were.
- **Recall** — of all the real positives, how many it actually found.
- **F1-score** — one balanced number combining precision and recall.
- **Confusion matrix** — a table of correct hits vs mistakes (we'll see it live).

When to use it + key points

Logistic Regression is the go-to **starting model** for yes/no (binary) classification.

Great because

- Simple, fast, interpretable
- Gives probabilities, not just labels
- Strong baseline for any classifier

Remember

- Scale your features first
- Works best on roughly linear splits
- Check recall when mistakes are costly

PART 2

Code Walkthrough

the script, explained one block at a time

Python

pandas

scikit-learn

Import the libraries



```
import pandas as pd
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import (accuracy_score,
                             classification_report, confusion_matrix)
```

What this block does

- **pandas** — handle data as a table.
- **load_breast_cancer** — a built-in dataset (no download!).
- **train_test_split, StandardScaler, LogisticRegression** — our ML tools.
- **metrics** — the report-card tools.

Load the dataset



```
data = load_breast_cancer()
```

What this block does

- Loads everything into **data**: the measurements, the correct answers, and the column names.
- It's a famous practice dataset — 569 tumors, 30 features each.

Separate features (X) and target (Y)



```
X = pd.DataFrame(data.data,  
                  columns=data.feature_names)  
Y = data.target
```

What this block does

- **X** = the input clues (radius, texture, smoothness...), as a neat table.
- **Y** = the correct labels (0 = malignant, 1 = benign).
- Naming the columns keeps everything readable.

Split into train & test



```
X_train, X_test, Y_train, Y_test = \
    train_test_split(
        X, Y,
        test_size=0.2,
        random_state=42)
```

What this block does

- Keep **80%** to train, hide **20%** to test.
- **random_state=42** makes the split the same every run (repeatable).
- We never test on data the model already saw.

Scale the features



```
scalar = StandardScaler()  
X_train = scalar.fit_transform(X_train)  
X_test  = scalar.transform(X_test)
```

What this block does

- **fit_transform** on train: learn the scale *and* apply it.
- **transform** only on test: reuse the same scale — no peeking.
- Now all features sit on a similar range.

Create & train the model



```
model = LogisticRegression()  
model.fit(X_train, Y_train)
```

What this block does

- Create an empty model — right now it knows nothing.
- **.fit()** is where the learning happens: it studies the clues + answers.
- It figures out the pattern separating malignant from benign.

Make predictions



```
predict = model.predict(X_test)
```

What this block does

- Give the trained model the **hidden test clues**.
- It returns a guess (0 or 1) for each tumor.
- These guesses go into **predict**.

Measure performance



```
print("ACCURACY SCORE",  
      accuracy_score(Y_test, predict)*100)  
print(classification_report(  
      Y_test, predict))
```

■ OUTPUT

```
ACCURACY SCORE 97.368...  
precision recall f1  
malignant 0.98 0.95 0.96  
benign 0.97 0.99 0.98  
accuracy 0.97
```

Reading the result

- **Accuracy** $\approx$ **97.4%** — nearly all 114 test tumors classified correctly.
- **Recall (benign) 0.99** — it caught almost every benign case.
- High precision + recall $\rightarrow$ a trustworthy little model.

Visualize with a confusion matrix



```
cm = confusion_matrix(Y_test, predict)
disp = ConfusionMatrixDisplay(
    confusion_matrix=cm,
    display_labels=data.target_names)
disp.plot(cmap=plt.cm.Blues)
plt.show()
```

What this block does

- Builds the matrix of correct vs confused guesses.
- **Diagonal** = correct; **off-diagonal** = mistakes.
- Then draws it as a colored picture (next page!).

RESULT

The confusion matrix, explained

Out of 114 test tumors: **41 + 70 = 111 correct**. Only **3 mistakes** — 2 malignant missed as benign, and 1 benign flagged as malignant.

Confusion Matrix — 97.37% accuracy on 114 test tumors

ACTUAL	PREDICTED	
	malignant	benign
malignant	41 True Malignant	2 Missed (→benign)
benign	1 False Alarm	70 True Benign

REFERENCE

The full script

```
import pandas as pd
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

data = load_breast_cancer()
X = pd.DataFrame(data.data, columns=data.feature_names)
Y = data.target

X_train, X_test, Y_train, Y_test = train_test_split(X, Y, test_size=0.2, random_state=42)

scalar = StandardScaler()
X_train = scalar.fit_transform(X_train)
X_test = scalar.transform(X_test)

model = LogisticRegression()
model.fit(X_train, Y_train)
predict = model.predict(X_test)

print("ACCURACY SCORE", accuracy_score(Y_test, predict)*100)    # 97.368...
print(classification_report(Y_test, predict))
```

RECAP

Logistic Regression in one minute

- It's a **classification** algorithm — predicts a probability, then a class.
- The **sigmoid** turns the linear score into a 0–1 probability; 0.5 is the cut-off.
- **Scale** features and **split** train/test before training.
- Grade with accuracy, precision, recall, f1, and the confusion matrix.
- On the breast-cancer data it hit **~97.4% accuracy** — only 3 errors out of 114.

Made with ♥ by Nikita Mishra · DIGITAL GYAAN